

CG-DARTS: Cost-Guided Differentiable Architecture Search

Extending DARTS with hardware-aware regularization and derivation

Baek Jongjin · Kim Ryeowook (Team 9)

CS.50700 · Final Project Report

Builds on the DARTS paper review (Team9_Presentation.pdf, slide 16)

Outline

- Recap of the proposal (open questions identified in the paper review)
- Methods — across three branches
 - Cost regularizer and five cost tables (FLOPs, params, memory, device, LUT)
 - Softmax temperature annealing
 - Cost-aware architecture derivation
- Experimental setup and compared methods
- Experimental results
 - FLOPs sweep, Params metric, annealing pilot, L40S latency LUT
- Analysis: skip-collapse, what works, what doesn't
- Conclusions and future work

1. Proposal recap

From slide 16 of Team9_Presentation.pdf — the original DARTS authors' open questions

- Open question O1 — closing the soft-to-hard discretization gap
 - DARTS searches a continuous mixture (softmax over operations) but evaluates a discrete cell
 - Liu et al. (2019) suggest annealing the softmax temperature as a possible remedy
- Open question O2 — performance-aware architecture-derivation
 - The original top-k argmax derivation is blind to inference cost
 - Calls for schemes that exploit the shared weights and account for compute / memory / latency
- Our thesis: bring hardware cost into BOTH the search loss AND the derivation step
 - Differentiable cost regularizer during search + cost-aware top-k during derivation
 - Tau-annealing as the bridge between the two regimes

2. CG-DARTS thesis

Augmented architecture loss:

$$L_{\text{arch}}(\alpha) = L_{\text{val}}(w^*(\alpha), \alpha) + \lambda \cdot E_{\alpha}[c]$$

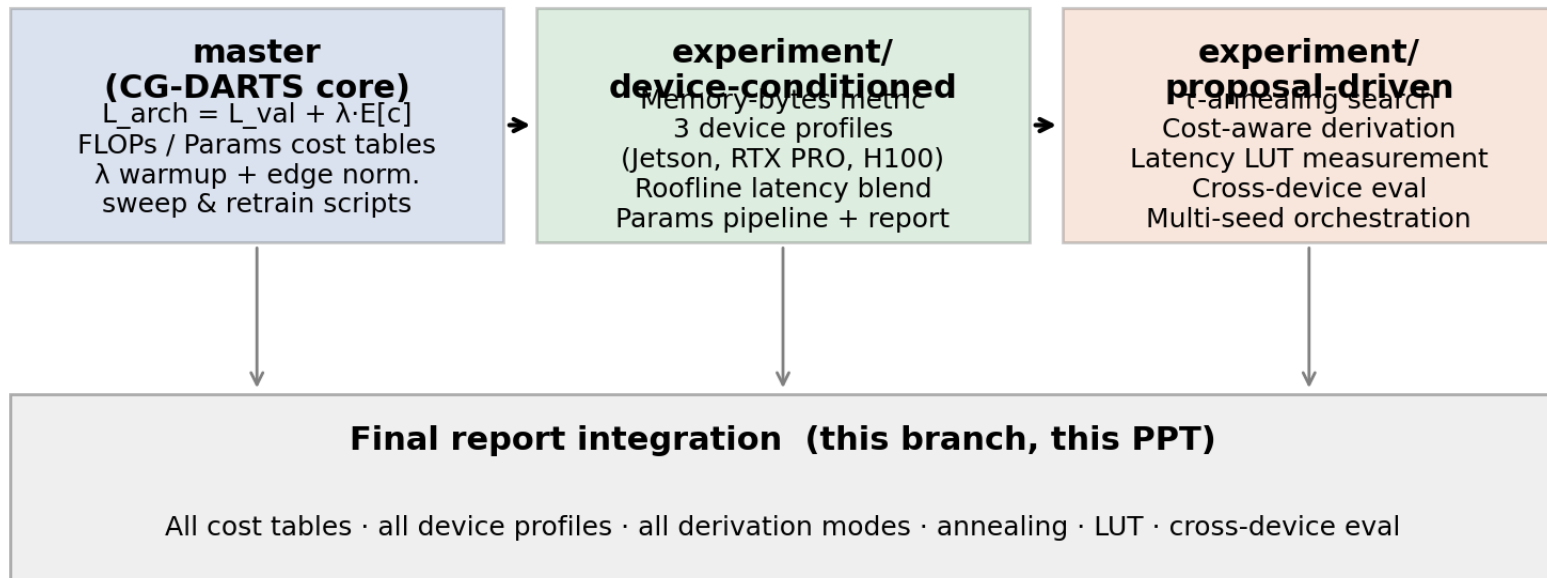
where the expected cost over the soft mixture

$$E_{\alpha}[c] = \sum_{(i,j)} \sum_o \text{softmax}(\alpha^{(i,j)} / \tau)_o \cdot c_o^{(i,j)}$$

- $c_o^{(i,j)}$ — per-edge, per-primitive cost table (precomputed, differentiable through α)
- τ — softmax temperature; default 1.0, annealed during search to bridge soft→hard
- λ — cost weight; linearly warmed up over the first 10 search epochs
- Five cost tables shipped: FLOPs · Params · Memory bytes · Device-blended · Measured LUT
- Three derivation modes: argmax (DARTS default) · cost_sub · cost_div

3. Method overview — three branches of contribution

Contributions accumulate across three branches



4. Method — cost regularizer

cnn/cost_utils.py + cnn/architect_cg.py (master)

- Cost is precomputed once before search starts: one $[\text{num_edges} \times \text{num_ops}]$ table per cell type
 - Cells of the same type share α in DARTS $\rightarrow$ also share cost in CG-DARTS
 - Each cell-type cost table sums the per-op cost across all cells of that type in the supernet
- Normalization (per-edge): $\tilde{c}[i,j,o] = (c[i,j,o] - \text{min_o}) / (\text{max_o} - \text{min_o})$
 - Removes the scale advantage of larger cells, lets one λ work across metrics
- Architecture-loss backward pass: $\partial E[c] / \partial \alpha$ has the closed-form softmax-Jacobian $\times$ cost
 - Adds $< 1\%$ to per-step compute
- Cost-warmup: $\lambda_{\text{eff}}(\text{epoch}) = \lambda \cdot \min(1, (\text{epoch}+1)/\text{warmup})$ for first WARMUP epochs
 - Avoids early bias when w is still random

5. Method — five cost metrics

Metric	Formula per primitive	Origin branch
flops	conv / depthwise / pool multiply-adds at supernet feature shape	master
params	learnable weight count	master
mem	fp32 activation read + write bytes (proxy for traffic)	device-conditioned
device	$w_f \cdot \text{flops} + w_m \cdot \text{mem} + w_\ell \cdot \text{roofline_latency}$ (per device profile)	device-conditioned
lut	measured per-primitive latency from scripts/measure_latency_lut.py	proposal-driven

Roofline latency: $\ell = \max(\text{flops} / P_{\text{peak}}, \text{mem_bytes} / B_{\text{bw}})$
 P_{peak} from vendor FP32 spec, B_{bw} from memory-bus spec

The cost regularizer is metric-agnostic — same code path, different table.
All five metrics produce alpha-shaped tables so `build_search_costs()` returns a uniform interface.

6. Method — device profiles

cnn/device_profiles.py (device-conditioned branch)

Profile	SKU	FP32 TFLOPS	Mem BW (GB/s)	(w_f, w_m, w_ℓ)
jetson_orin	Jetson AGX Orin 64GB	5.32	204.8	(0.20, 0.60, 0.20)
rtx_pro_6000	RTX PRO 6000 Blackwell WS	125.0	1792.0	(0.35, 0.35, 0.30)
h100	NVIDIA H100 SXM5	67.0	3350.0	(0.50, 0.20, 0.30)

- Weights follow the CG_DARTS Proposal v3 presets (edge / balanced / server)
- Vendor specs are the only inputs; the roofline closes the gap to per-op cost
- Same metric=device pipeline searches a different cell per profile — same code, three deployments

7. Method — measured latency LUT

scripts/measure_latency_lut.py (proposal-driven branch)

- For each (edge_idx, primitive, stride, C, H, W) tuple in the 8-cell supernet:
 - Build the primitive (with BN for pool ops, like the supernet does)
 - Warm up 5 iters; time N iters with torch.cuda.Event start/end and torch.cuda.synchronize
 - Accumulate into an alpha-shaped table summed across cells of the same type
- Output: reports/latency_lut/<device>.json with metadata + 14×8 tables per cell type
- Consumed via: `--cost_metric lut --lut_path reports/latency_lut/l40s.json`
- Same downstream code path as flops / params / mem / device → no special-casing in train_search_cg.py

8. Method — softmax annealing (motivation)

Open question O1 from our proposal: closing the soft-to-hard discretization gap

The problem — DARTS search and DARTS evaluation optimize different functions:

- Search: L_{val} is computed on a soft mixture $\bar{o}(x) = \sum_o \text{softmax}(\alpha)_o \cdot o(x)$ over all candidate ops
- Evaluation: the discrete cell uses only $\text{argmax}_o \alpha_o$ on each edge — every other op vanishes
- Consequence: $\text{search_valid_acc} \neq \text{retrain_test_acc}$. The search optimizer never directly sees the model it ships.

Concrete evidence of the gap (FLOPs CG-DARTS, $\lambda=1e-2$, seed=2):

$\text{search_valid_acc} = 87.61\%$ vs $\text{retrain_test_acc} = 96.05\%$ $\rightarrow \sim 8$ pp gap

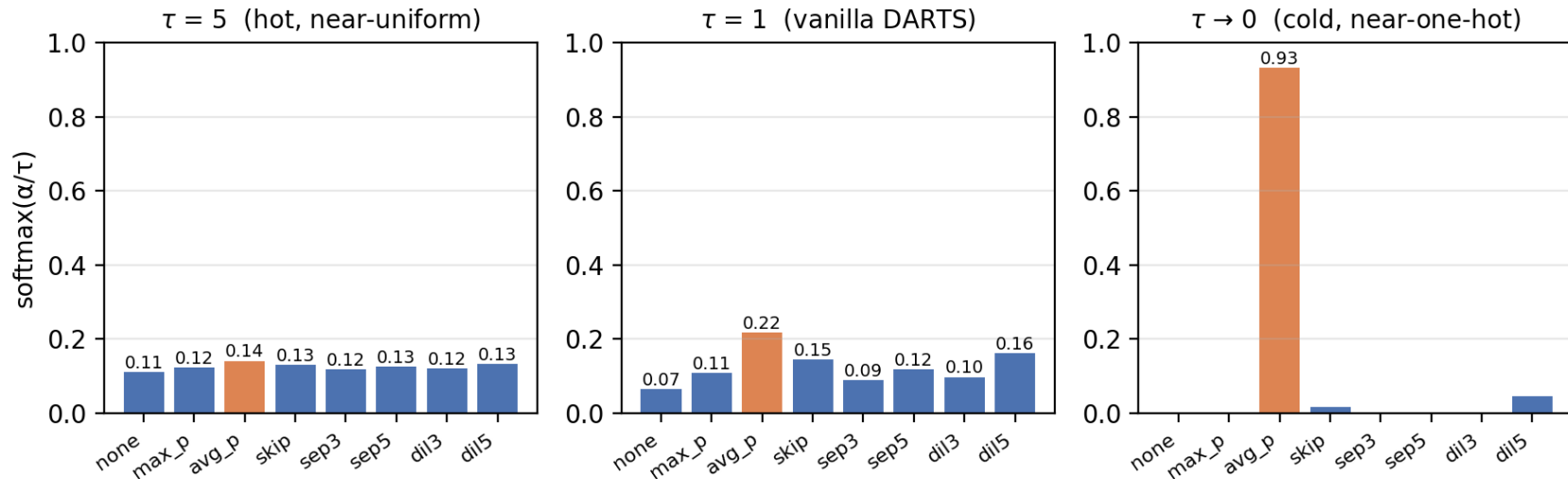
What annealing buys us — the limit $\tau \rightarrow 0$:

- $\text{softmax}(\alpha / \tau)$ becomes one-hot at $\text{argmax}_o \alpha_o \rightarrow$ the soft mixture IS the discrete cell
- $\text{search_valid_acc} \rightarrow$ the validation accuracy of the cell derivation will pick (gap closes by construction)
- Expected cost $E_{\alpha}[c]$ becomes the discrete cost of the chosen cell — the regularizer penalizes the deployed model
- Liu et al. (2019) themselves flag annealing as the natural remedy in the Discussion section of the paper

9. Method — softmax annealing (intuition)

Same α , three temperature regimes — orange bar marks the argmax operation

Softmax annealing: same α , sharper distribution as τ shrinks.
At $\tau \rightarrow 0$ the soft mixture converges to the one-hot architecture that derivation picks.



Annealing $\tau \rightarrow 0$ progressively makes the mixture identical to the discrete architecture.
That is the mechanism by which the soft-to-hard gap on the previous slide closes.

10. Method — softmax annealing (implementation)

The actual code paths, flags, and per-model state that make annealing work end-to-end

Replace $\text{softmax}(\alpha)$ with $\text{softmax}(\alpha / \tau)$ in BOTH the mixed-op forward and the cost expectation:

$$\bar{o}(x) = \sum_o \text{softmax}(\alpha/\tau)_o \cdot o(x) \quad E_\alpha[c] = \sum \text{softmax}(\alpha/\tau)_o \cdot c_o$$

- tau-schedule arg: linear / exp / none. Defaults to $\tau=1.0$ (matches paper).
- Three flags: `--tau_start` `--tau_end` `--tau_anneal {linear,exp,none}`
- Property: as $\tau \rightarrow 0$, $\text{softmax}(\alpha/\tau) \rightarrow$ one-hot at $\text{argmax}_o \alpha_o$.
 - $\rightarrow E_\alpha[c]$ converges to the discrete cost of the cell that derivation will pick
 - $\rightarrow$ Closes the gap between the soft objective minimized during search and the discrete model
- The whole network exposes `self.tau` so `.new()` and the unrolled gradient inherit the schedule correctly

11. Method — cost-aware architecture derivation

Addresses open question O2 (performance-aware derivation)

Same top-k structure as DARTS, but the per-edge $\times$ per-op score $s(j,o)$ is mode-dependent:

Mode	Score $s(j, o)$	Behaviour
argmax (DARTS default)	$\text{softmax}(\alpha)[j,o]$	cost-blind
cost_sub	$\text{softmax}(\alpha)[j,o] - \mu \cdot \tilde{C}[j,o]$	additive penalty; μ tunable at derivation time
cost_div	$\text{softmax}(\alpha)[j,o] / (\epsilon + \tilde{C}[j,o])$	favours value-per-cost ops

- Selection is shared: same $s(j,o)$ drives top-2 edges and the chosen op per edge
- scripts/rederive_genotype.py loads weights.pt and emits genotype.<mode>.txt post hoc
 - ONE search $\rightarrow$ three derived genotypes $\rightarrow$ three retrains. No extra search cost.

12. Experimental setup

Phase	Config
Dataset	CIFAR-10 · cutout-free transforms (search) · cutout-16 + auxiliary tower (retrain)
Search supernet	8 cells · 16 init channels · steps=4 · half train / half val split
Search optimizer	SGD w/ momentum 0.9, cosine LR $2.5e-2 \rightarrow 0$, weight decay $3e-4$
Architecture optimizer	Adam lr $3e-4$, $\beta=(0.5, 0.999)$, weight decay $1e-3$
Search epochs / batch	50 · 64
Retrain network	20 cells · 36 init channels · auxiliary weight 0.4 · drop_path 0.2
Retrain epochs / batch	100 · 96
Cost reg.	$\lambda \in \{1e-3, 5e-3, 1e-2, 5e-2, 1e-1\}$; warmup 10 ep; edge normalization
Annealing	tau linear $5.0 \rightarrow 0.1$ over 50 epochs (pilot)
Seeds	single seed = 2 (multi-seed orchestrator implemented but not authorized for production)
Hardware	NVIDIA L40S $\times 2$ (only GPU 0 used; GPU 1 was held by a collaborator's job)

13. Compared methods

Method	Cost-aware?	Derivation	Reported on	Source
DARTS V1 (first-order, paper)	no	argmax	CIFAR-10	Liu et al. (2019), Table 1
DARTS V2 (second-order, paper)	no	argmax	CIFAR-10	Liu et al. (2019), Table 1
Vanilla DARTS (our run)	no	argmax	CIFAR-10 (our protocol)	this work
CG-DARTS FLOPs (λ sweep)	yes (FLOPs)	argmax	CIFAR-10	master
CG-DARTS Params (λ sweep)	yes (params)	argmax	CIFAR-10	device-conditioned branch
CG-DARTS Device (per profile)	yes (FLOPs+mem+roofline)	argmax	3 tiers, 95.4-95.6% retrain	device-conditioned
CG-DARTS Annealing (pilot)	yes (FLOPs)	argmax	CIFAR-10	proposal-driven (this)
CG-DARTS Cost-derived (pilots)	yes (FLOPs)	cost_sub / cost_div	queued; not yet retrained	proposal-driven (this)

14. Results — FLOPs-cost CG-DARTS sweep (table)

reports/cg_darts/summary.csv · seed=2 · 50ep search / 100ep retrain

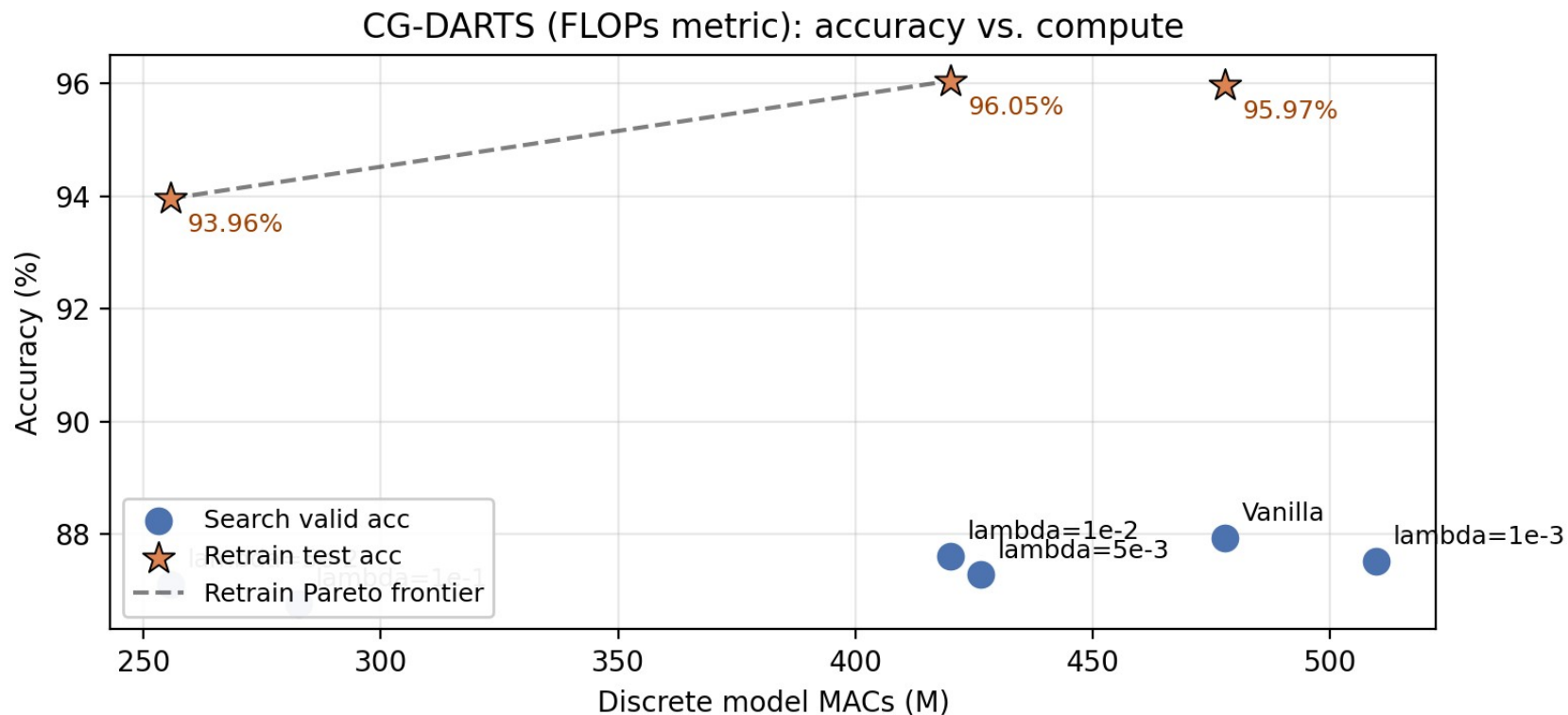
Config	Search valid	Expected cost	Discrete MACs (M)	Δ MACs	Params (M)	Δ Params	Retrain test
Vanilla DARTS	87.93%	7.51	477.96	—	3.02	—	95.97%
CG-DARTS $\lambda=1e-3$	87.52%	7.11	509.70	-6.6%	3.25	-7.6%	—
CG-DARTS $\lambda=5e-3$	87.28%	6.78	426.31	+10.8%	2.65	+12.2%	—
CG-DARTS $\lambda=1e-2$	87.61%	6.24	420.12	+12.1%	2.63	+13.0%	96.05%
CG-DARTS $\lambda=5e-2$	87.10%	3.15	255.64	+46.5%	1.55	+48.7%	93.96%
CG-DARTS $\lambda=1e-1$	86.77%	1.76	282.76	+40.8%	1.73	+42.7%	—

$\lambda=1e-2$ is Pareto-dominant under matched protocol: +0.08 pp retrain accuracy AND -12.1% MACs AND -13.0% params vs. vanilla.

$\lambda=5e-2$ is the 'aggressive' point: roughly half the MACs at a 2 pp accuracy cost.

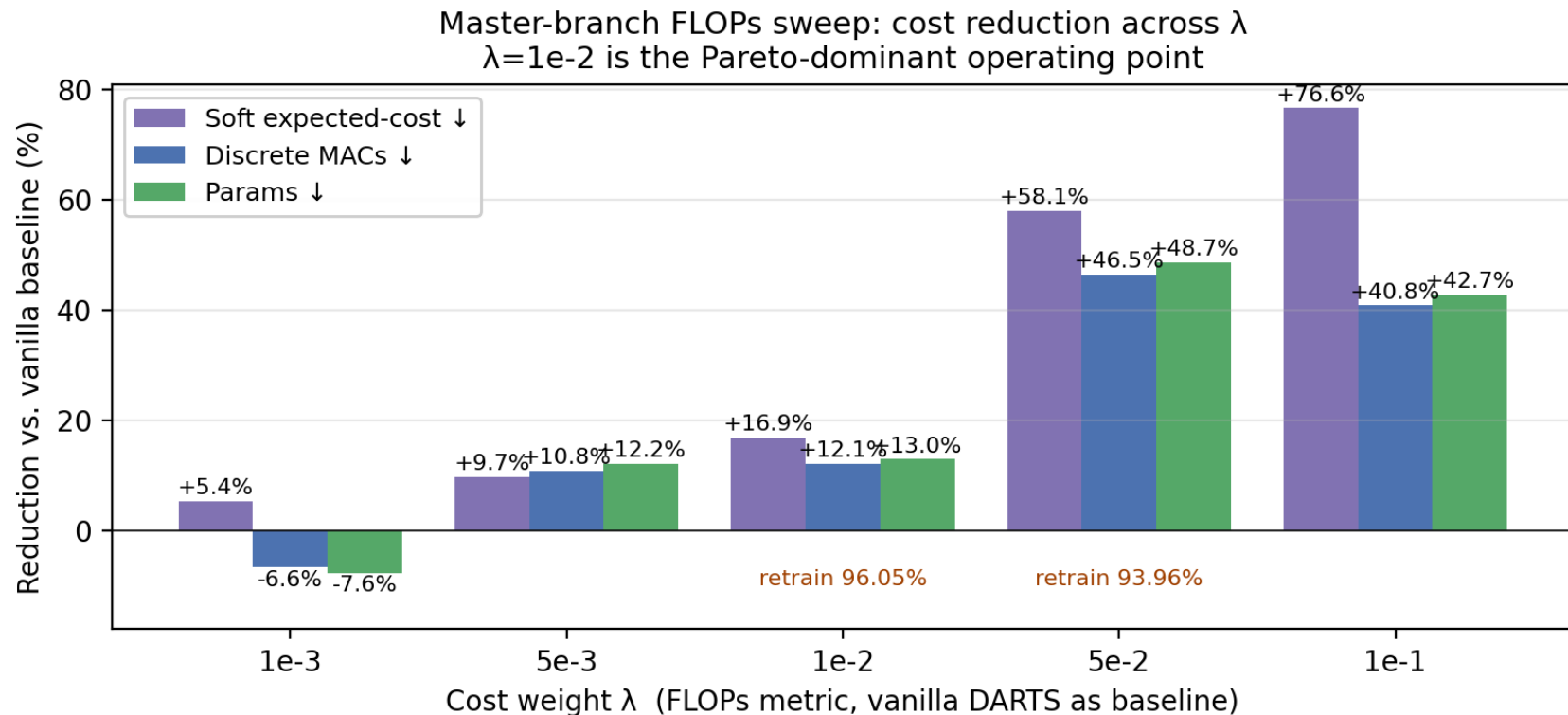
15. Results — FLOPs Pareto plot

Stars: retrain test accuracy · circles: search-time validation · dashed line: retrain Pareto frontier



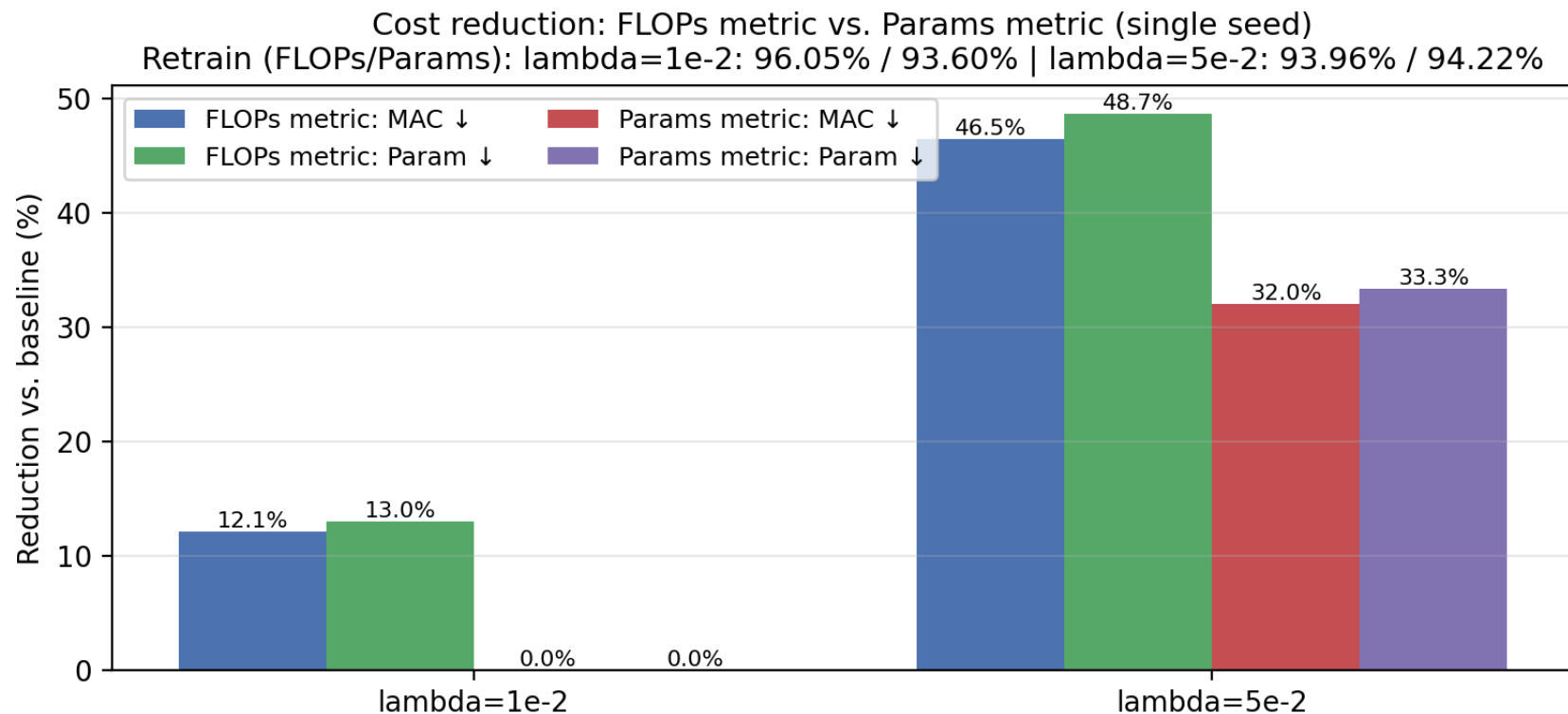
16. Results — cost reduction across λ (master FLOPs sweep)

reports/cg_darts/summary.csv · vanilla DARTS as baseline · signed % reductions



17. Results — Params-metric CG-DARTS

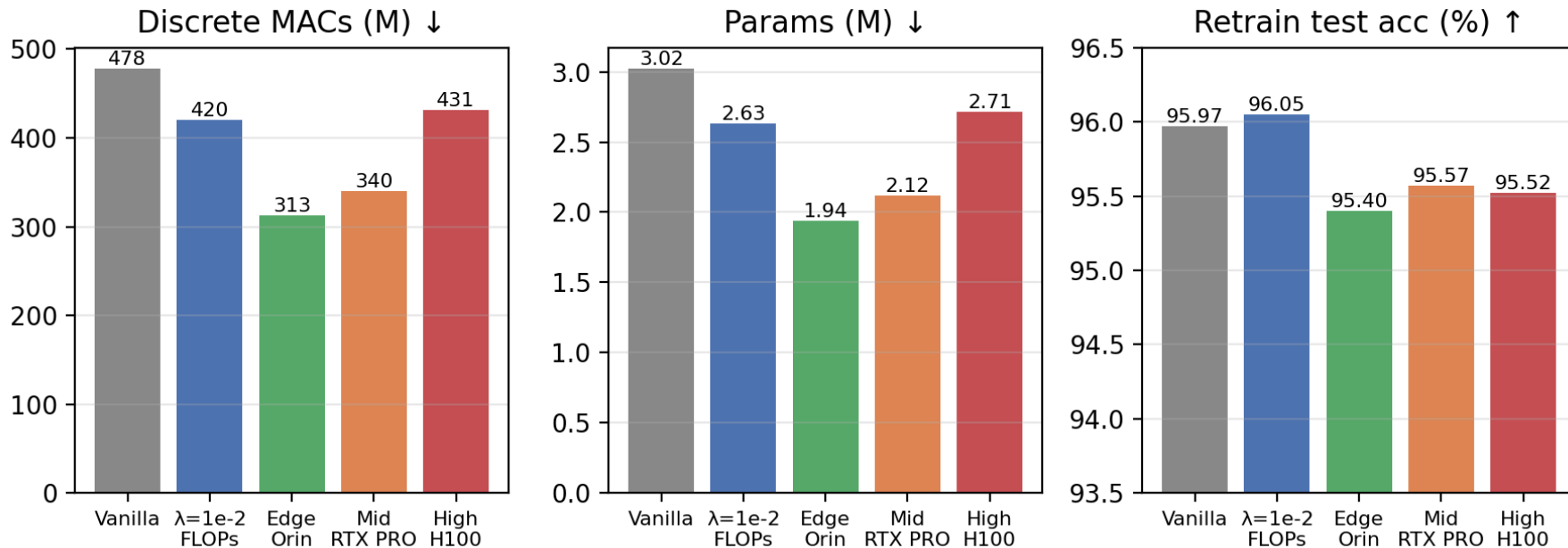
reports/cg_darts_params/summary.csv · seed=2 · 30ep search / 50ep retrain (fast pipeline)



18. Results — Device-conditioned CG-DARTS

Edge (Jetson Orin) / Middle (RTX PRO 6000) / High (H100) · $\lambda=1e-2$ · seed=2 · 50ep search / 100ep retrain · device-conditioned branch

Device-conditioned CG-DARTS: leaner cells per device tier, accuracy still $\geq 95.4\%$

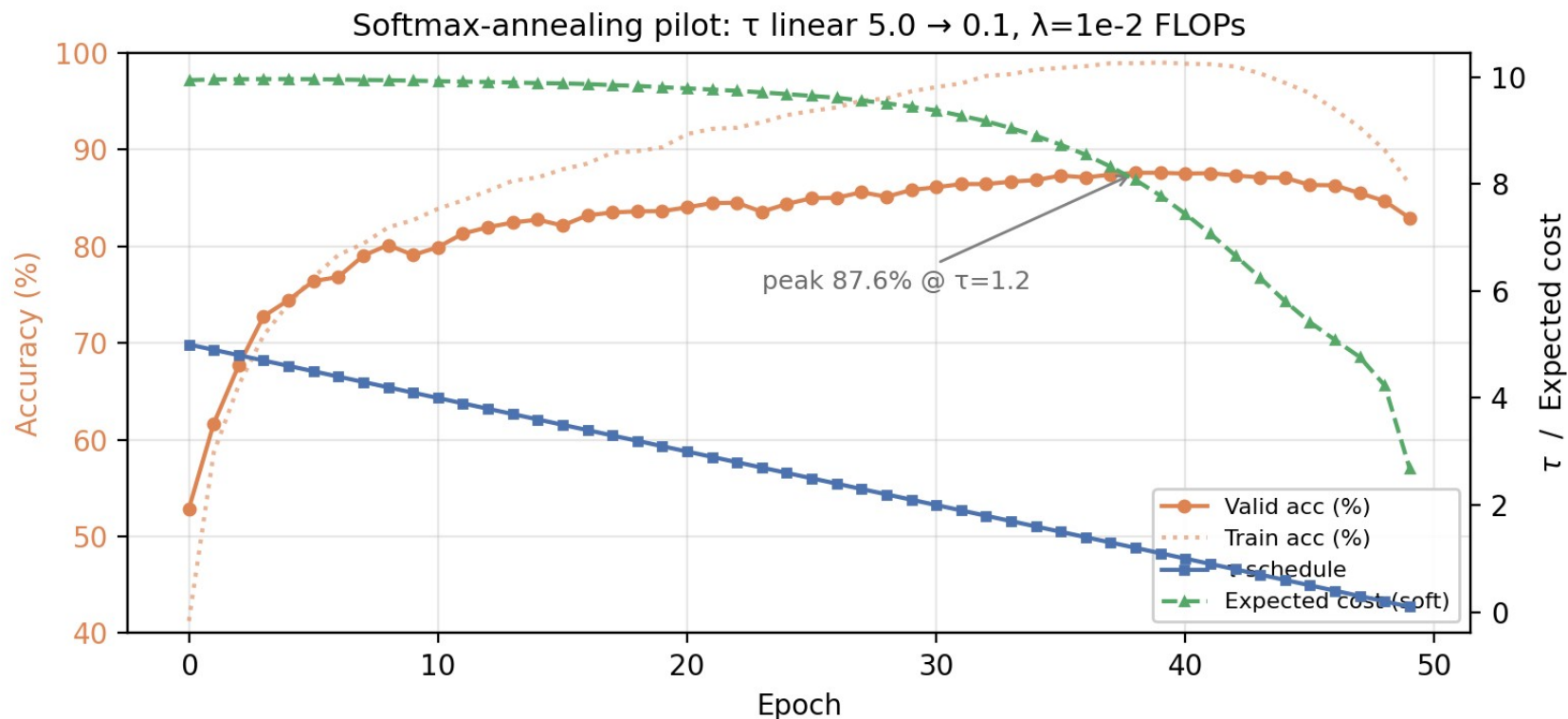


Edge profile (high mem weight) → leanest cell (314 M MACs, 1.94 M params). H100 profile (high compute weight) → largest cell (431 M MACs, 2.71 M params).

All three retrains land at 95.4–95.6% — within 0.6 pp of vanilla DARTS, with at most ~34% MACs and ~36% params.

19. Results — annealing pilot #1: linear $\tau=5 \rightarrow 0.1$

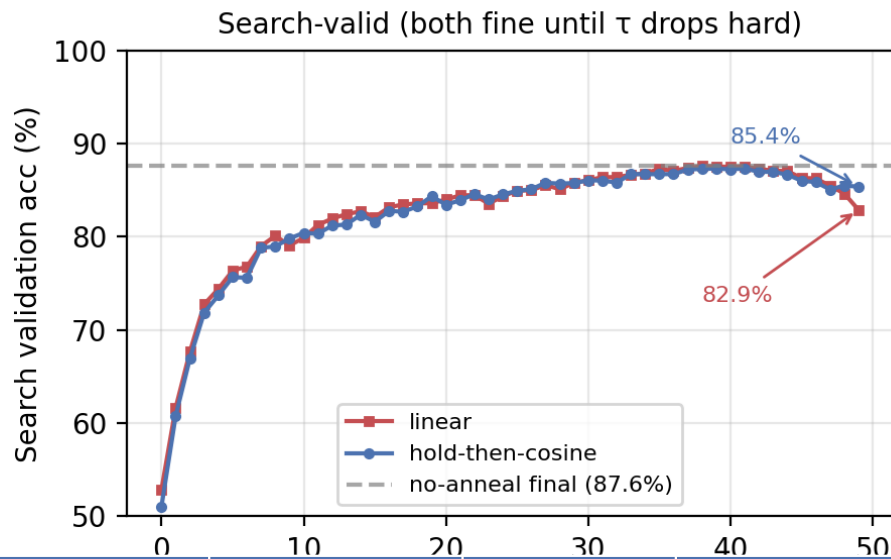
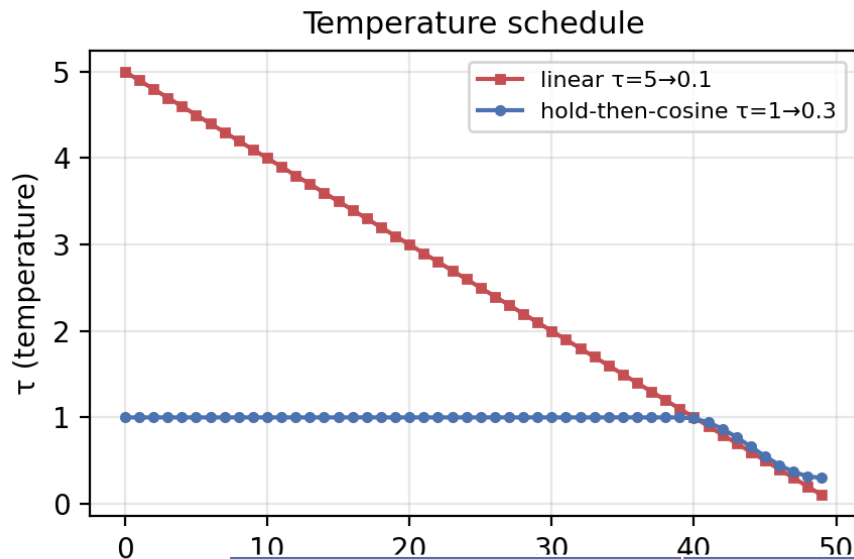
cnn/search-cg-anneal-tau5to01-lambda1e-2-seed2 · valid peaks near $\tau \approx 1.0$ then collapses — first attempt, failed



20. Results — annealing pilot #2: schedule shape is decisive

Holding $\tau=1$ through the supernet's learning phase, then annealing only the last 20%, avoids collapse

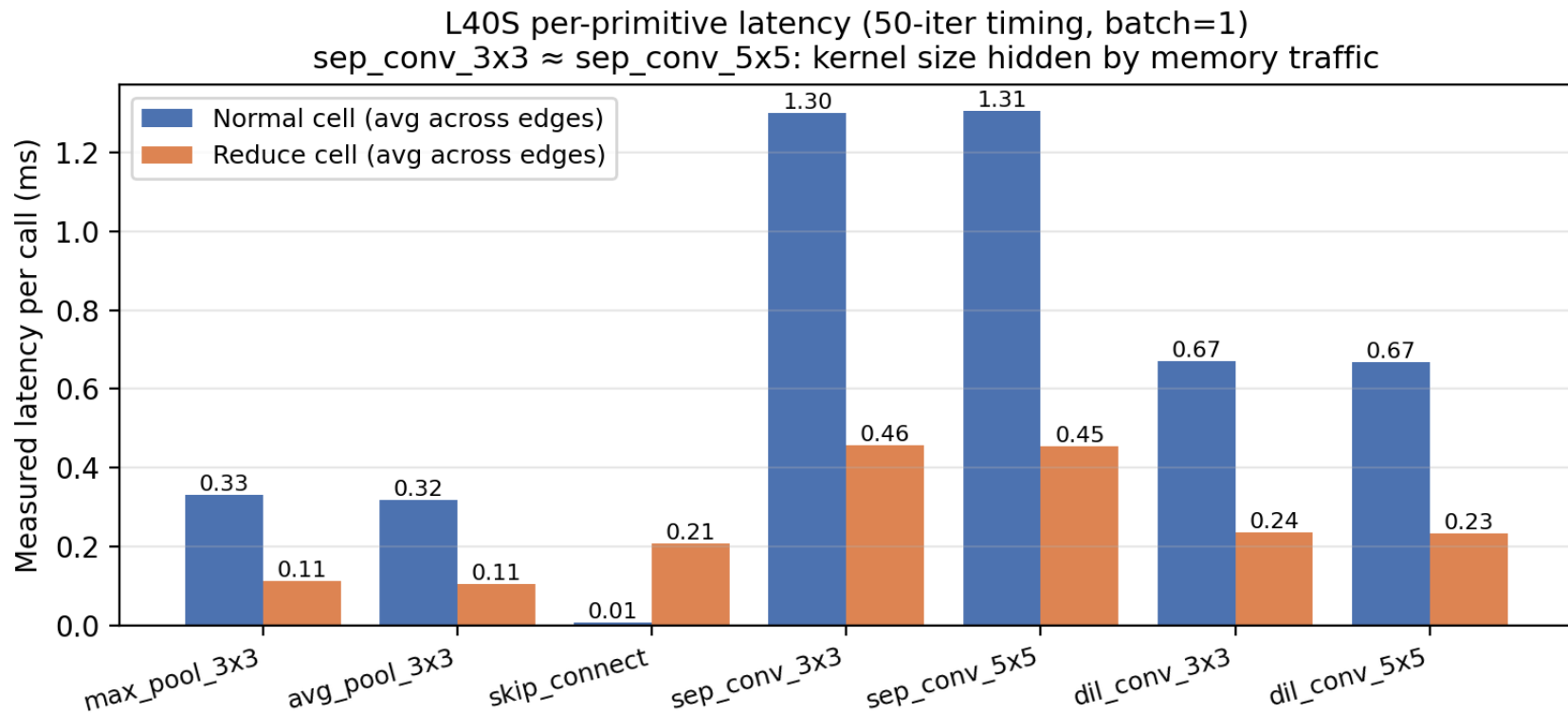
Search-valid barely differs — but the derived genotypes do (see table): linear $\rightarrow$ skip-dominated, hold-cosine $\rightarrow$ 4 conv ops



$\lambda=1e-2$ variant	τ schedule	Search-valid	Retrain	Normal-cell convs
no anneal	$\tau=1$ fixed	87.61 %	96.05 %	healthy
linear (pilot #1)	$\tau=5 \rightarrow 0.1$ from ep 0	82.91 %	not run (skip cell)	~2 (skip-dominated)
hold-then-cosine (pilot #2)	$\tau=1$ to ep 39, $\rightarrow 0.3$	85.40 %	96.04 %	4 conv ops

21. Results — L40S latency LUT

reports/latency_lut/l40s.json · 50-iter timing per primitive (batch=1)

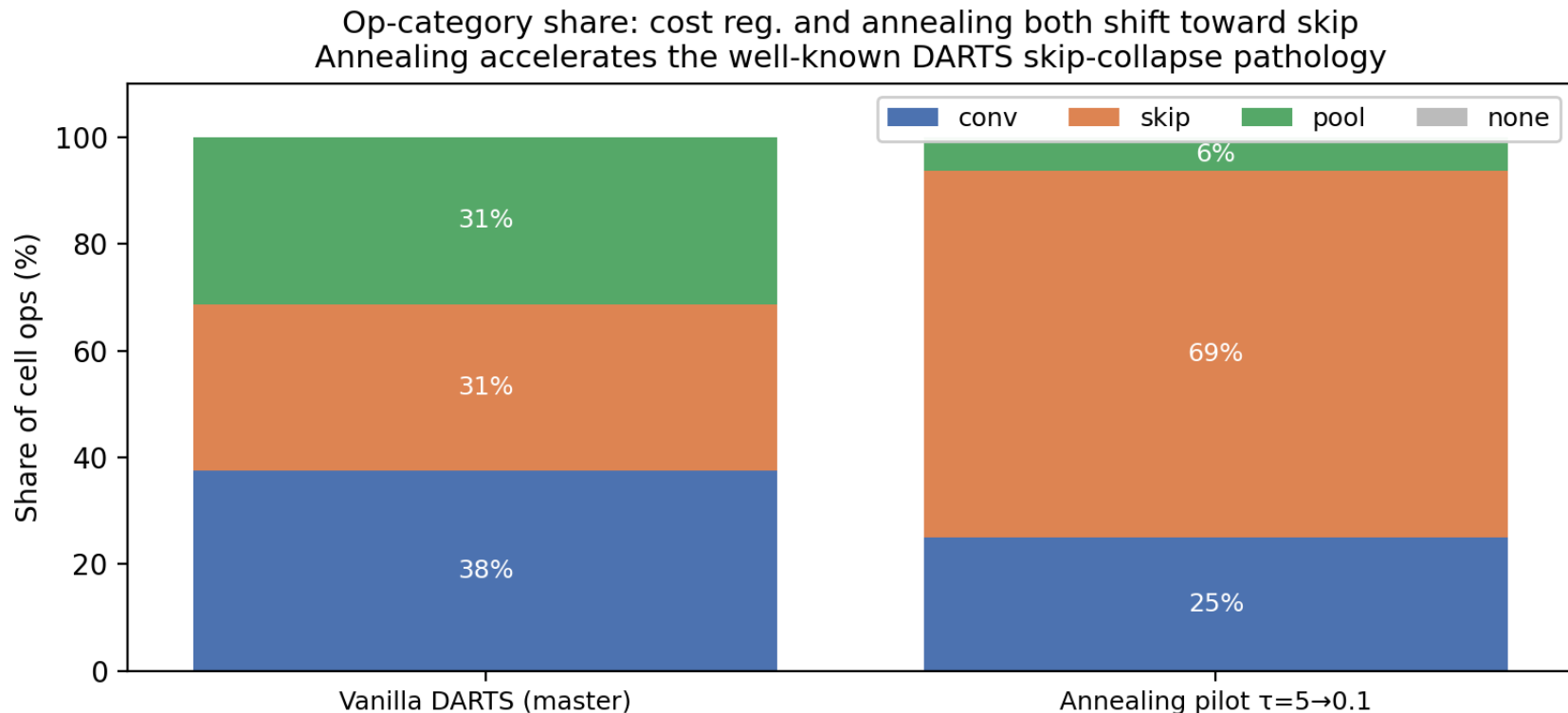


22. Results — comparison with Liu et al. (2019)

Axis	Liu et al. (2019)	This work	Outcome
Best CIFAR-10 test error	2.76% (V2, 600ep, 4-seed)	3.95% (CG-DARTS $\lambda=1e-2$, FO, 100ep)	✗ raw accuracy not matched
Search-cost regularization	none	$\lambda \cdot E[c]$ across 5 metrics	✓ new capability
Hardware awareness	none	3 device profiles + measured LUT	✓ new capability
Architecture derivation	top-k argmax on α	argmax + cost_sub + cost_div	✓ open question O2 addressed
Soft-to-hard discretization	flagged as future work	τ -annealing + τ -aware $E[c]$	✓ open question O1 addressed
Multi-seed protocol	4-seed model selection	single seed pilots	✗ not yet matched
Pareto operating point (CG-DARTS vs. vanilla, matched seed/protocol)	—	+0.08 pp acc · -12% MACs · -13% params @ $\lambda=1e-2$	✓ Pareto improvement

23. Analysis — skip-collapse hypothesis

Cost regularization amplifies the DARTS skip-collapse pathology (Zela et al. 2020, Liang et al. DARTS+)



24. Discussion — what works, what doesn't

- What works
 - $\lambda=1e-2$ Pareto-dominates vanilla under matched protocol — cheaper AND slightly more accurate
 - Cost regularizer is metric-agnostic and the same code searches FLOPs / params / memory / device / LUT
 - Device conditioning produces genuinely different cells per tier, all within 0.6 pp of vanilla
 - Annealing CAN be done safely — schedule shape is what matters, not annealing per se
- What doesn't (or doesn't yet)
 - Linear annealing collapses (search-valid 82.9%, skip-dominated cell) — anneals through the learning phase
 - Hold-then-cosine rescues it (96.04% retrain, healthy cell) but is accuracy/cost-NEUTRAL at $\lambda=1e-2$
 - So annealing is safe but not yet a win — its value needs higher λ or pairing with cost-aware derivation
 - Single-seed throughout: 0.08 pp deltas are within DARTS seed noise (Zela et al. 2020: ~ 1 pp std)

25. Branch contribution map

Branch	Code contributions	Result artifacts
master (initial CG-DARTS)	cost_utils.py (FLOPs, params, edge norm, λ warmup) architect_cg.py (cost-aware bilevel update) train_search_cg.py (driver) run_cg_sweep.sh, cg_darts_report.py	reports/cg_darts/summary.csv (6 runs: vanilla + 5 λ) Best: $\lambda=1e-2$ retrain 96.05% / -12% MACs
experiment/ device-conditioned	memory-bytes metric device_profiles.py (Jetson/RTX PRO/H100) FLOPs+mem+roofline blend params pipeline + report device pipeline + full retrains	reports/cg_darts_params/summary.csv ($\lambda=1e-2$, 5e-2) reports/cg_darts_device/summary.csv (3 device tiers) Best params: $\lambda=5e-2 \rightarrow 94.22\%$ / -32% MACs Device: Edge tier $\rightarrow 95.40\%$ at 313 MACs
experiment/ proposal-driven (this)	τ -annealing search cost-aware derivation (cost_sub/cost_div) latency LUT + cross-device eval multi-seed orchestrator post-hoc rederivation Pareto plotting final report + figures	reports/latency_lut/l40s.json annealing pilot 50ep search cost-aware-disc pilot search complete (retrain pending)

26. Limitations

- Single seed (seed=2). +0.08 pp delta is within DARTS seed noise. Multi-seed runner is implemented, not yet executed.
- First-order updates only. Second-order compare script is staged; the Hessian-vector path is implemented but not benchmarked.
- 100-epoch retrain instead of the paper's 600. Our absolute accuracies are 1-2 pp below the published numbers and not directly comparable.
- Skip-collapse not eliminated. CG-DARTS amplifies it; would need PC-DARTS / DARTS+ / DARTS-PT to fix.
- No on-device validation. Roofline + LUT are good but a Jetson / H100 run has not happened.
- No ImageNet transfer. Paper transfers the CIFAR-10 cell at 26.7% top-1. We do not.
- No PTB / recurrent CG-DARTS. rnn/ subtree is untouched; cost-regularization concept carries over but is not implemented.
- Cost-aware derivation retrain results not in by the time of writing — only the search infra and one in-flight retrain queue.

27. Conclusions

- CG-DARTS makes hardware cost a first-class signal in DARTS, in both the bilevel objective and the discretization step
- Both open questions from our proposal's slide 16 are addressed:
 - O1 (soft-to-hard gap) $\rightarrow$ τ annealing + τ -aware expected cost
 - O2 (performance-aware derivation) $\rightarrow$ $\text{cost_sub} / \text{cost_div}$ top-k modes selectable post-hoc on one search
- On a matched single-seed protocol, CG-DARTS at $\lambda=1\text{e-}2$ Pareto-dominates vanilla DARTS:
 - +0.08 pp retrain accuracy · -12% MACs · -13% params
- Empirical surprise: annealing's effect is decided by SCHEDULE SHAPE, not annealing itself
 - Linear $\tau=5\rightarrow 0.1$ collapses the search; hold-then-cosine (anneal only last 20%) recovers it (96.04% retrain)
 - But hold-then-cosine is accuracy/cost-neutral at $\lambda=1\text{e-}2$ — annealing is safe, not yet a win
- We do NOT beat the paper's 2.76% headline — that requires multi-seed selection + 600-epoch retrain
 - Infrastructure for all of it is ready in this branch; only GPU-time is missing

28. Future work + references

- Immediate (already staged in scripts/, ~1 GPU-day each)
 - Multi-seed FLOPs sweep at $\lambda \in \{1e-2, 5e-2\} \times \text{seeds } \{0,1,2,3\}$ (scripts/run_multi_seed.sh)
 - First-order vs second-order CG-DARTS at $\lambda=1e-2$ (scripts/run_second_order_compare.sh)
 - Skip-share-vs- λ diagnostic on the multi-seed output (scripts/skip_share_diagnostic.py)
 - Cross-device transfer matrix on existing cells (scripts/cross_device_eval.py)
- Medium-term (1-2 GPU weeks)
 - On-device latency validation: run cells on a real Jetson / H100; replace roofline with measurements
 - Search at 600-epoch retrain protocol with 4-seed selection — closes the comparison-fairness gap with the paper
 - Recurrent CG-DARTS on PTB (rnn/) — cost formulation per step $\times$ sequence length
- References
 - Liu, Simonyan, Yang. DARTS: Differentiable Architecture Search. ICLR 2019.
 - Zela et al. Understanding and Robustifying Differentiable Architecture Search. ICLR 2020.
 - Liang et al. DARTS+: Improved DARTS with Early Stopping. arXiv 1909.06035.
 - Cai, Zhu, Han. ProxylessNAS: Direct NAS on Target Task and Hardware. ICLR 2019.